

Implementationsdokumentation

Audio I/O & 3D-Raumdaten

Kais

5. Juli 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Architektur-Übersicht	3
2	Audio I/O Handling	4
2.1	Anforderungen (Akzeptanzkriterien)	4
2.2	Theoretischer Hintergrund	4
2.2.1	WAV-Format und Audio-Digitalisierung	4
2.2.2	Clipping und Normalisierung	4
2.3	Implementierung: <code>wav_handler.py</code>	4
2.3.1	WAV-Einlesung	4
2.3.2	WAV-Export mit Normalisierung	5
2.3.3	Integration in die Pipeline	5
3	3D-Raumdaten: OBJ-Parser	6
3.1	Theoretischer Hintergrund	6
3.1.1	OBJ-Format	6
3.1.2	Dreiecks-Repräsentation	6
3.2	Implementierung: <code>obj_parser.py</code>	6
3.3	Export ins JSON-Format	7
4	Integration und Testing	8
4.1	End-to-End-Pipeline	8
4.2	Abhängigkeiten	8
5	Mathematische Modelle	9
5.1	Convolution im Frequenzraum	9
5.2	Ray-Tracer Intersection (Möller-Trumbore)	9
6	Akzeptanzkriterien und Validierung	10
6.1	Ticket 3: Audio I/O	10
6.2	Raumdaten: OBJ-Parser	10
7	Zusammenfassung und Ausblick	11
7.1	Implementierte Features	11
7.2	Mögliche Erweiterungen	11
A	Code-Referenz	12
A.1	<code>wav_handler.py</code>	12
A.2	<code>obj_parser.py</code>	12
A.3	<code>main.py</code>	12

Kapitel 1

Einführung

Dieses Dokument beschreibt die Implementierung zweier kritischer Komponenten des Distributed Acoustic Ray Tracers:

1. **Audio I/O (Ticket 3):** Einlesen trockener WAV-Dateien und Export des verarbeiteten, halligen Audio-Signals mit Clipping-Prävention
2. **Raumdaten (OBJ-Parser):** Laden von 3D-Modellen aus `.obj`-Dateien und Konvertierung in das interne JSON-Format

Die Implementierung erfolgt in Python und integriert sich nahtlos in die bestehende Pipeline, die Rust-basierte Physik-Engine, DSP-Convolution und Distributed-Computing-Frameworks verbindet.

1.1 Architektur-Übersicht

Das System folgt einer 4-stufigen Pipeline:

- [1/4] Rust Physics Engine: Simuliert Schallstrahlen, erzeugt Impulsantwort (IR)
- [2/4] Audio-Einlesung: Lädt trockene WAV-Datei
- [3/4] DSP-Convolution: Wendet IR auf Audio an
- [4/4] Audio-Export: Normalisiert und speichert Ergebnis

Die Raumdaten werden parallel in den Tracer geladen, um realistische 3D-Geometrien zu unterstützen.

Kapitel 2

Audio I/O Handling

2.1 Anforderungen (Akzeptanzkriterien)

1. Sauberes Einlesen der `unprocessed.wav` ohne Qualitätsverlust
2. Export der finalen WAV-Datei ohne Artefakte
3. Normalisierung, damit geschichtetes Audio beim Export nicht clippt

2.2 Theoretischer Hintergrund

2.2.1 WAV-Format und Audio-Digitalisierung

Das WAV-Format speichert Audiodaten als zeitdiskrete Samples mit konstanter Abtastfrequenz f_s (typisch 44.1 kHz oder 48 kHz):

$$x[n] = \int_{(n-0.5)/f_s}^{(n+0.5)/f_s} x_{\text{analog}}(t) dt$$

Jedes Sample liegt im Bereich $[-1.0, 1.0]$ (bei normalisierten 16/32-Bit PCM).

2.2.2 Clipping und Normalisierung

Wenn während der Convolution (Faltung) die Amplitude $|y[n]|$ über 1.0 steigt, tritt **Clipping** auf:

$$y_{\text{clipped}}[n] = \begin{cases} 1.0 & \text{wenn } y[n] > 1.0 \\ y[n] & \text{wenn } |y[n]| \leq 1.0 \\ -1.0 & \text{wenn } y[n] < -1.0 \end{cases}$$

Um dies zu vermeiden, normalisieren wir:

$$y_{\text{norm}}[n] = \frac{y[n]}{A_{\text{max}}} \quad \text{wobei} \quad A_{\text{max}} = \max_n |y[n]|$$

2.3 Implementierung: `wav_handler.py`

2.3.1 WAV-Einlesung

```

1 import numpy as np
2 import soundfile as sf
3
4 def read_wav(wav_path: str) -> tuple:
5     """
6     Reads a WAV file and returns the audio data as a NumPy array
7     along with the corresponding sample rate.
8     """
9     audio_data, sample_rate = sf.read(wav_path)
10    return audio_data, sample_rate

```

Funktionsweise:

- Die soundfile-Bibliothek liest WAV-Dateien robust
- audio_data ist ein NumPy-Array der Form (samples,) oder (samples, channels)
- sample_rate ist die Abtastfrequenz in Hz

2.3.2 WAV-Export mit Normalisierung

```

1 def write_wav(wav_path: str, audio_data, sample_rate: int) -> None:
2     """
3     Normalizes the audio data to prevent clipping and exports it to a WAV file.
4     """
5     max_amplitude = np.max(np.abs(audio_data))
6     if max_amplitude > 1.0:
7         audio_data = audio_data / max_amplitude
8     sf.write(wav_path, audio_data, sample_rate)

```

Algorithmus:

Algorithm 1 Normalisierter WAV-Export

```

 $A_{\max} \leftarrow \max_n |y[n]|$ 
if  $A_{\max} > 1.0$  then
     $y[n] \leftarrow y[n]/A_{\max}$  für alle  $n$ 
end if
Schreibe normalisierte Samples in WAV-Datei

```

2.3.3 Integration in die Pipeline

```

1 # main.py Schritt [2/4]: Audio laden
2 dry_audio, sample_rate = read_wav(input_wav_path)
3
4 # Schritt [3/4]: Convolution
5 wet_audio = convolve(dry_audio, sample_rate, ir_data)
6
7 # Schritt [4/4]: Normalisieren und exportieren
8 write_wav(output_wav_path, wet_audio, sample_rate)

```

Kapitel 3

3D-Raumdaten: OBJ-Parser

3.1 Theoretischer Hintergrund

3.1.1 OBJ-Format

Das Wavefront .obj-Format ist ein textbasiertes 3D-Modellformat:

```
1 # Vertices (Ecken)
2 v 1.0 2.0 -1.5 # Vertex 1
3 v 2.0 2.0 -1.5 # Vertex 2
4 v 1.0 3.0 -1.5 # Vertex 3
5
6 # Faces (Dreiecke)
7 f 1/1/1 2/2/1 3/3/1
```

Indexierung: OBJ nutzt 1-basierte Indizes, Python 0-basierte – Umrechnung nötig.

3.1.2 Dreiecks-Repräsentation

$$\text{Triangle} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{bmatrix}$$

3.2 Implementierung: obj_parser.py

```
1 def load_obj_to_triangles(file_path: str) -> list:
2     vertices = []
3     triangles = []
4     path = Path(file_path)
5     if not path.exists():
6         raise FileNotFoundError(f"The 3D file '{file_path}' was not found.")
7     with path.open('r') as f:
8         for line in f:
9             line = line.strip()
10            if line.startswith('v '):
11                parts = line.split()
12                x, y, z = float(parts[1]), float(parts[2]), float(parts[3])
13                vertices.append([x, y, z])
14            elif line.startswith('f '):
15                parts = line.split()
16                v1_idx = int(parts[1].split('/')[0]) - 1
17                v2_idx = int(parts[2].split('/')[0]) - 1
18                v3_idx = int(parts[3].split('/')[0]) - 1
```

```
19         triangle = [vertices[v1_idx], vertices[v2_idx], vertices[v3_idx]]
20         triangles.append(triangle)
21     return triangles
```

3.3 Export ins JSON-Format

```
1 import json
2 triangles = load_obj_to_triangles("room.obj")
3 data = {
4     "triangles": triangles,
5     "triangle_count": len(triangles)
6 }
7 with open("room_geometry.json", "w") as f:
8     json.dump(data, f, indent=2)
```

Kapitel 4

Integration und Testing

4.1 End-to-End-Pipeline

```
1 print("--- Starting Acoustic Reverb Pipeline ---")
2 ir_data = run_rust_and_load_json(rust_binary_path, json_path)
3 dry_audio, sample_rate = read_wav(input_wav_path)
4 wet_audio = convolve(dry_audio, sample_rate, ir_data)
5 write_wav(output_wav_path, wet_audio, sample_rate)
6 print("--- Pipeline Complete! ---")
```

4.2 Abhängigkeiten

Modul	Bibliotheken	Version
wav_handler.py	soundfile, numpy	Any
obj_parser.py	pathlib	Built-in
convolution.py	scipy, numpy	Any

Kapitel 5

Mathematische Modelle

5.1 Convolution im Frequenzraum

$$Y(f) = X(f) \cdot H(f), \quad y[n] = \mathcal{F}^{-1}\{Y(f)\}$$

5.2 Ray-Tracer Intersection (Möller-Trumbore)

Gegeben Strahl $\vec{r}(t) = \vec{o} + t\vec{d}$ und Dreieck $(\vec{a}, \vec{b}, \vec{c})$:

$$t = \frac{(\vec{c} - \vec{a}) \times (\vec{b} - \vec{a}) \cdot (\vec{a} - \vec{o})}{(\vec{c} - \vec{a}) \times (\vec{b} - \vec{a}) \cdot \vec{d}}, \quad \tau = \frac{t}{343 \text{ m/s}}, \quad n = \lfloor \tau \cdot f_s \rfloor$$

Kapitel 6

Akzeptanzkriterien und Validierung

6.1 Ticket 3: Audio I/O

Kriterium	Erfüllt?	Test
Sauberer Einlesen .wav	✓	<code>len(audio_data) > 0</code>
Export ohne Qualitätsverlust	✓	Hörtest, Spektrum-Analyse
Normalisierung gegen Clipping	✓	<code>max(abs(audio)) <= 1.0</code>

6.2 Raumdaten: OBJ-Parser

Kriterium	Erfüllt?	Test
Parser liest .obj	✓	<code>len(triangles) > 0</code>
JSON-Export	✓	Gültige JSON, Geometrie-Metadaten

Kapitel 7

Zusammenfassung und Ausblick

7.1 Implementierte Features

1. **Audio I/O:** Robust WAV-Lesen/Schreiben mit soundfile
2. **Clipping-Prävention:** Peak-Normalisierung vor Export
3. **OBJ-Parser:** Liest Vertices/Faces, konvertiert in Dreiecke
4. **Pipeline-Integration:** Nahtlose Verkettung aller Komponenten

7.2 Mögliche Erweiterungen

1. **Multi-Channel Audio:** Stereo/5.1 Surround Support
2. **Material-Attribute:** Pro-Dreieck Material-IDs aus OBJ
3. **Adaptive Normalisierung:** Loudness-basierte Pegel (LUFS) statt Peak
4. **Geometrie-Optimierungen:** BVH-Tree für schnelle OBJ-Intersects

Anhang A

Code-Referenz

A.1 wav_handler.py

Code-Referenz: `python_service/wav_handler.py`

A.2 obj_parser.py

Code-Referenz: `python_service/obj_parser.py`

A.3 main.py

Code-Referenz: `python_service/main.py`